Labirentte yol bulan ajan

Statik aramadan değişen engellere uyum sağlayan bir sisteme

Duvarları çiz. En kısa yolu bul. Harita değişince rotayı güncelle.

20 × 50 grid

1.000 karelik çalışma alanı

Hibrit planlama

BFS + artımlı D* Lite

Görsel karşılaştırma

A* ve BFS kare sayıları

Projenin amacı ve kuralları

Eşit maliyetli, dört yönlü hareket üzerinde en kısa yol

Harita ve etkileşim

20 satır × 50 sütun
9 geçitli engel sütunu; 162 duvar
Fareyle duvar ekleme ve silme

Arama ve hareket

Başlangıç → hedef
Yukarı / aşağı / sağ / sol
Her geçişin maliyeti: 1 adım

Başlangıç: (10, 1) Hedef: (10, 48) [satır, sütun; 0'dan başlayan indeks]

Harita: data/maze.json • Örnek harita üretimi: generate_maze.py

Uygulamanın yeni arayüzü

Kontrol paneli, canlı sayaçlar ve algoritma karşılaştırması



BFS ve A* nasıl çalışıyor?

İki yöntem de bu gridde en kısa yolu bulur; arama sıraları farklıdır.

BFS • Genişlik öncelikli arama

FIFO kuyruk: deque

Önce yakın katmanlar incelenir.
Hedef bulununca arama durur.
Eşit maliyetli gridde optimal yol.

A* • Hedefe yönelik arama

Öncelik kuyruğu: heapq
f(n) = g(n) + h(n)
g: başlangıçtan gelen maliyet
h: Manhattan uzaklığı

h(n) = |satır − hedef_satır| + |sütun − hedef_sütun|

Kod: algorithms.py → search(..., method="bfs") veya method="astar"

Hibrit yöntemde iki algoritma birleşiyor

BFS’nin oluşturduğu bilgi, D* Lite’ın başlangıç durumu olur.

01 İlk plan

Hedeften ters BFS
Tam mesafe alanı
g ve rhs tablolarını başlat

02 Harita değişimi

Ajanın konumunu koru
Değişen duvarları belirle
İlgili komşuları güncelle

03 D* Lite onarımı

Önceki bilgiyi kullan
Gerekli maliyetleri düzelt
Yeni en kısa yolu çıkar

Akış: HybridPlanner.plan() → DStarLite.replan() → repair() → path()

BFS / A* düğmeleri sıfırdan arar; hibrit düzenleme sırasında eski durumu korur.

İlk haritada ölçülen sonuçlar

Aynı başlangıç, aynı hedef ve aynı duvarlar

Algoritma	İncelenen kare	Yol / adım
BFS	802	155
A*	747	155
Hibrit	838	155

A*: aynı en kısa yol, BFS’ye göre 55 kare daha az inceleme.

Hibrit ilk planda tam mesafe alanı kurar; tekil BFS hedefe ulaşınca durur.
Ölçüt: plan başına işlenen farklı kare sayısı; süre veya toplam işlem sayısı değildir.

Harita değiştiğinde ne oluyor?

Aynı değişiklik dizisinde, aynı mevcut konumdan karşılaştırma

Senaryo	BFS	A*	Hibrit
Ajan 3 adım ilerledi	802	747	0
Yola engel eklendi	801	746	6
Engel silindi	802	747	5
Geçit tamamen kapandı	428	428	429
Geçit yeniden açıldı	802	747	320

0 kare: mevcut mesafe alanıyla yol çıkarılabildi; sıfır işlem süresi değildir.

Hibritin avantajı değişikliğe bağlıdır. Geçit kapandığında üç yöntem de yol bulamaz.

Projenin dosya yapısı

Arama mantığı, kullanıcı olayları ve görsel tasarım ayrı tutulur.

Dosya	Görevi
algorithms.py	BFS, A*, D* Lite ve hibrit planlayıcı
main.py	Fare / klavye olayları ve animasyon akışı
ui.py	Kontrol paneli, sayaçlar ve sonuç tablosu
generate_maze.py / data/maze.json	Örnek haritanın üretimi ve verisi
benchmark.py	Aynı senaryoların CSV / grafik karşılaştırması
test_algorithms.py / test_ui.py	Doğruluk ve kullanıcı etkileşimi testleri

Teknik ek • Temel arama fonksiyonları

algorithms.py: geçerli komşular → arama → sonuç

neighbors(cell, walls, rows, cols)

algorithms.py

Grid dışını ve duvarları eleyerek geçerli dört komşuyu üretir.

Kullanım: BFS, A* ve D* Lite komşu kareleri bu fonksiyonla gezer.

search(start, goal, walls, ..., method)

algorithms.py

BFS veya A* çalıştırır; parent kayıtlarıyla en kısa yolu yeniden oluşturur.

Çıktı: Result(path, expanded, discovered). Yol yoksa path boş listedir.

manhattan(a, b)

algorithms.py

İki karenin satır ve sütun farklarının mutlak değerlerini toplar.

D* Lite öncelik anahtarında kullanılır; search() içinde aynı sezgi yerel olarak hesaplanır.

Teknik ek • Hibrit planı yöneten fonksiyonlar

Planlayıcı aynı arama bilgisini çağrılar arasında saklar.

HybridPlanner.plan(...)

algorithms.py

İlk çağrıda ters BFS başlatır; sonraki çağrılarda D* Lite ile yeniden planlar.

Hedef veya grid boyutu değişirse motor yenilenir; diğer güncellemelerde korunur.

HybridPlanner.reset()

algorithms.py

Motoru, plan sayısını ve toplam inceleme istatistiklerini sıfırlar.

Kullanım: yeni arama başlatılırken veya örnek haritaya dönülürken.

DStarLite.__init__(...)

algorithms.py

Hedeften BFS ile mesafe alanını hesaplar; g ve rhs değerlerini eşit başlatır.

Bu mesafe alanı ilk yolun çıkarılmasında ve sonraki artımlı onarımlarda kullanılır.

Teknik ek • D* Lite yolunu nasıl onarır?

g: tutulan maliyet rhs: komşular üzerinden hesaplanan aday maliyet

DStarLite.replan(start, walls)

algorithms.py

Değişen duvarları ve komşularını belirler; ajan konumunu ve km değerini günceller.

Sıra: update_vertex() → repair() → path().

DStarLite.update_vertex(u)

algorithms.py

rhs değerini yeniler; g ≠ rhs olan kareyi onarım kuyruğuna ekler.

Duvar için sonsuz maliyet, hedef için sıfır maliyet kullanılır.

DStarLite.repair()

algorithms.py

Öncelik sırasıyla gerekli tutarsız maliyetleri düzeltir.

Mevcut başlangıcın yolu için yeterli koşullar sağlandığında durur.

DStarLite.path()

algorithms.py

Hedefe maliyeti düşük komşuları izleyerek güncel rotayı çıkarır.

Başlangıç erişilemezse boş liste döndürür; döngü oluşmasını denetler.

Teknik ek • Kullanıcıdan algoritmaya

main.py: bir düğme veya fare olayı planlamayı tetikler.

App.begin(mode, replan=False)

main.py

Aramayı çalıştırır, karşılaştırma sonuçlarını hazırlar ve animasyonu başlatır.

replan=True ise ajan konumu ve önceki hibrit motor korunur.

App.edit(event)

main.py

Fare konumunu kareye çevirir; duvar ekler veya siler.

Başlangıç, hedef ve ajan korunur. Hibrit aktifse otomatik yeniden planlanır.

App.set_tool(tool)

main.py

Sol fare düğmesinin duvar çizme veya silgi işlevini seçer.

D: duvar, E: silgi. Sağ tık seçili araçtan bağımsız olarak duvar siler.

Teknik ek • Animasyonun aşamaları

Arama sonucu hesaplanır, ardından adımlar sırayla gösterilir.

search

İşlenen kareleri göster
Hız: kare / animasyon adımı
Mor arama alanı

path

Bulunan yolu sırayla çiz
Başlangıçtan hedefe
Sarı rota

walk

Ajanı rota üzerinde ilerlet
Hedefte: done
Yol yoksa: blocked

App.tick(_)

main.py

FuncAnimation zamanlayıcısında arama, yol ve hareket aşamalarını ilerletir.

Duraklatılmışsa işlem yapmaz. Son aşamada ajan hedefe ulaşır.

App.pause(): dur / devam • App.reset(): örnek haritaya dön • App.clear(): duvarları sil

Teknik ek • Ekranı oluşturan fonksiyonlar

ui.py tasarımı, main.py veri ve uygulama durumunu yönetir.

build_ui(app, COLORS, ROWS, COLS)

ui.py

Labirenti, kontrol panelini, sayaçları, düğmeleri ve karşılaştırma tablosunu kurar.

App.__init__ içinde bir kez çağrılır; düğmeleri ilgili olay fonksiyonlarına bağlar.

refresh_ui(app)

ui.py

Sayaçları, durum mesajını, seçili aracı ve algoritma satırının vurgusunu günceller.

App.refresh() üzerinden her görsel güncellemede çağrılır.

App.pixels() ve App.refresh()

main.py

Durumu renk matrisine dönüştürür; görüntüyü ve tablodaki sayıları yeniler.

Duvarlar arama renklerinin üstünde kalır; başlangıç, hedef ve ajan ayrıca işaretlenir.

Doğrulama ve projenin sınırları

Doğru yol, dinamik güncelleme ve kullanıcı etkileşimi ayrı ayrı denetlenir.

13 otomatik test

Rastgele haritalarda yol doğruluğu
Engel ekleme, silme ve geçit açma
Fare, silgi, duraklatma, sayaçlar
Duvarların görünür kalması

Ne iddia ediyoruz?

Anlık haritada en kısa yol
Önceki bilgiyle yeniden planlama
Her senaryoda daha hızlı değil
Kare sayısı, süre ölçümü değil

Çalıştırma: python -m unittest discover -v

Karşılaştırma: python benchmark.py → docs/results.csv ve docs/comparison.png

Canlı demo • 2–3 dakikalık akış

Önce sabit harita, ardından dinamik engel değişimi

1 Aç

Proje klasöründe: py main.py

2 Karşılaştır

BFS ve A* çalıştır; aynı yol uzunluğu, farklı kare sayılarını göster.

3 Değiştir

Hibriti başlat; ajan ilerlerken önüne duvar çiz.

4 Yeniden yol bul

D* Lite güncellemesini izle; silgiyle geçidi tekrar aç.

BFS ilk bilgiyi kurar. D* Lite değişeni onarır. A* karşılaştırma sağlar.